Kutty Banerjee

710 Saratoga Avenue Apt#U304 San Jose, CA 95129
Phone: (925)-209-7204 E-Mail: kutty.banerjee@gmail.com

Experience

Senior System Software Engineer, NVIDIA [2005 June ~ Present]

- **Project Denver – ARM CPU JIT**
  - Denver uses a **Trace JIT** called MTS to convert ARM instructions to native instructions. I maintain the Branch Profiling and Translation Cache code.
  - Implemented branch-to-assert and lazy assert-to-branch logic. Boosts SPECINT by upto 15%
  - Implemented Adaptive Branch Profiling framework to route hot translations to interpreter for a small time to regather branch weights and retranslate if weights shift.
  - Implementing logic to keep Translation Cache coherent with ARM Self Modifying Code
  - Implemented various stubs to profile transitions to and from interpreted / optimized code
- **Android Stack Perf Optimizations for Denver CPU**
  - **Linux Kernel** : Implemented Large Page Suport for ARMv7 to automatically coalesce contig small pages into 64K pages. 5% IPC boost on SPECFP
  - **Linux Kernel** : Implemented Thread Local Memory type in kernel. TLB invalidates to this memory are not broadcast to other cores. Patched Mozilla to use this memory type. Helps IPC by 20% on Google V8 Benchmark
  - Implemented Memory Profiler Extension to QEMU to study page access patterns for apps.
- **Project Orion – Cache Coherent CPU/GPU on a single SoC**
  - Designed and implemented proof of concept changes to AARCH64 Linux kernel and GPU kernel **MMU** layers to present a Unified Address Space between CPU and GPU. Also added asynchronous GPU page fault handling. This project has been cancelled.
- **Fermi Architecture GPU Kernel Tech Lead**
  - Drove the GPU kernel team from POR Architecture through RTL/Emulation/Bringup and Release.
- **Fermi GPU Thread Scheduling Lead**
  - Designed and implemented 85% of the GPU kernel **Thread Scheduling** code. This unit was redesigned from scratch in Fermi to avoid latency problems of the previous generation.
  - Implemented a Pipelined Scheduling Mode that allowed queues tasks to make progress with prefetch, TLB and cache warmup before threads are actually scheduled.
  - Implemented separate Scheduling Policies to saturate PCIE b/w for Graphics threads and Minmize Launch Latency for CUDA/PhysX/Video threads.
  - Designed the Error Recovery Subsystem and Power Aware Scheduling Subsystem. Mentored team members to implement the actual code.
  - Debugged perf and latency bottlenecks.
- **Fermi GPU Kernel MMU / Memory Management**
  - Helped the MMU lead wire up memory management layer of the Fermi GPU kernel. This includes basic support for Multi level Paga Table setup code, TLB invalidation logic, TLB invalidation batching optimizations, NOMMU mode support, fixed bugs!
- **Deterministic Verification of Asynchronous / ISR Code**
  - Designed and implemented a framework to allow ISR code to add probes in the code. Probes are patched at runtime with arbitrary wait logic to force hard to reproduce event conditions in a controlled deterministic way.
- **Non Stall Interrupts**
  - Implemented low latency lockless interrupt code path and ported all GPU notifiers to use this. This interrupt line is very rarely disabled and has low dependency on non ISR code.
- **Curie Architecture GPU Kernel**
  - Debugged all parts of the GPU kernel, fixed bugs and triaged perf bottlenecks.

Education

MS Computer Science, Worcester Polytechnic Institute, MA [2003 Aug ~ 2005 May]

- **Thesis**: Remote Execution for 3D Graphics. A distributed 3D Rendering pipeline (RMESA) to offload floating point intensive operations to a cluster.
- Recipient of **WPI Institute Fellowship Award** for outstanding research work.
- http://www.pseudoparallel.com/publications 4 Conference 1 Journal publication

BS Computer Science, University of Mumbai [1999 July ~ 2003 May]

Side Projects [http://www.pseudoparallel.com/]

Mostly focussing on Concurrency, Parallelism, Locks

- **Khronos (C):** An AutoParallelizing Query Processor that converts Serial C-like code into Parallel Instructions for multicore execution. Extracts Map-Reduce, Data Parallelism, Task Parallelism, I/O Pipelining automatically.
- **Task Parallel Libraries**: **(JAVA)** Special focus on Dynamic Parallelism and reducing lock contention on the requeue task logic. Also supports Pipelined Locks, Pipelined dependencies.
- **Concurrency Utilities / Locks (C/Java):** Implemented a large variety of locks with special focus on reduced CAS/fence instructions and fairness.
- **Task Parallel Library (C)** : Special focus on low latency low lock contention CAS/fenceless intra thread communication. Wired up to aide Project Khronos above.